



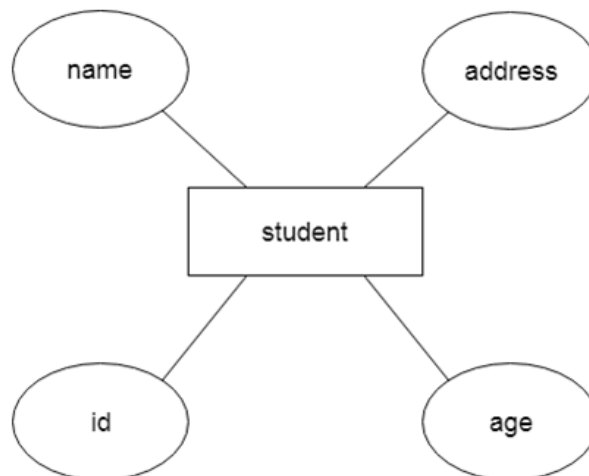
DBMS: MODULE-2

▼ Database Design Using ER model

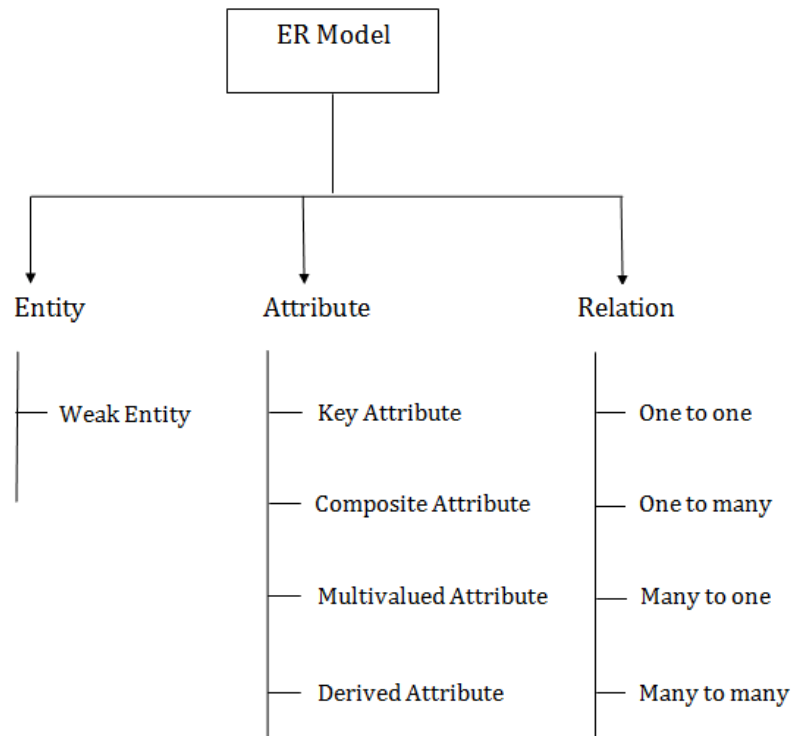
▼ Overview of the design process

- ER model stands for an Entity-Relationship model. It is a high-level data model. This model is used to define the data elements and relationship for a specified system.
- It develops a conceptual design for the database. It also develops a very simple and easy to design view of data.
- In ER modeling, the database structure is portrayed as a diagram called an entity-relationship diagram.

For example, Suppose we design a school database. In this database, the student will be an entity with attributes like address, name, id, age, etc. The address can be another entity with attributes like city, street name, pin code, etc and there will be a relationship between them.

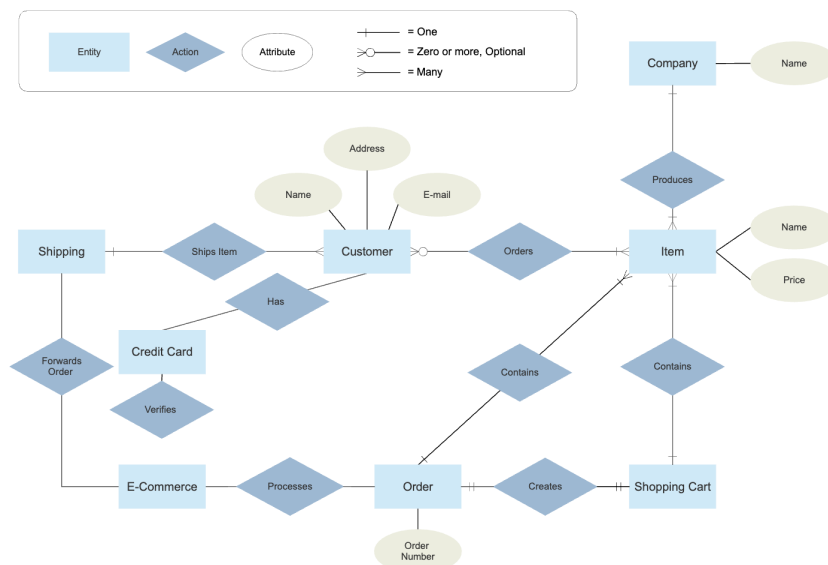


Components of ER Diagram



▼ Entity Relationship Model

Entity Relationship Diagram - Internet Sales Model



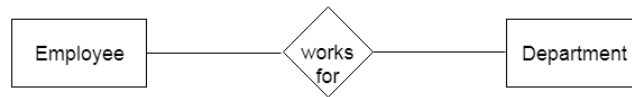
An Entity-Relationship (ER) model is a way to design a database by representing the entities (or objects) in the database and the relationships between those entities. An ER model is typically used to design a database at the conceptual level, before it is implemented in a specific database management system.

There are three main components in an **ER model**:

▼ Entities

An entity may be any object, class, person or place. In the ER diagram, an entity can be represented as rectangles.

Consider an organization as an example- manager, product, employee, department etc. can be taken as an entity.



a. Weak Entity

An entity that depends on another entity called a weak entity. The weak entity doesn't contain any key attribute of its own. The weak entity is represented by a double rectangle.



▼ Entity Set

An entity set is a set of entities of the same type that share the same properties. A database thus includes a collection of entity sets, each of which contains any number of entities of the same type.

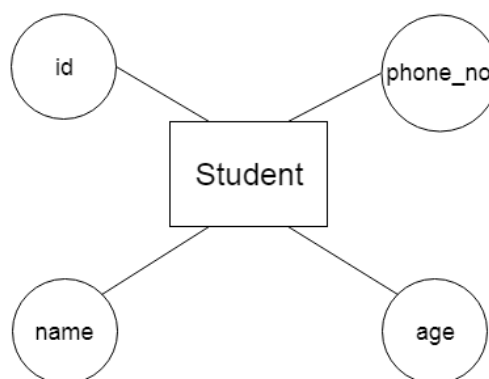


E-R diagram showing entity sets *instructor* and *student*.

▼ Attributes

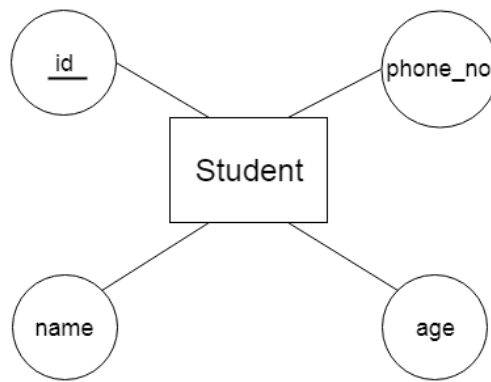
The attribute is used to describe the property of an entity. Eclipse is used to represent an attribute.

For example, id, age, contact number, name, etc. can be attributes of a student.



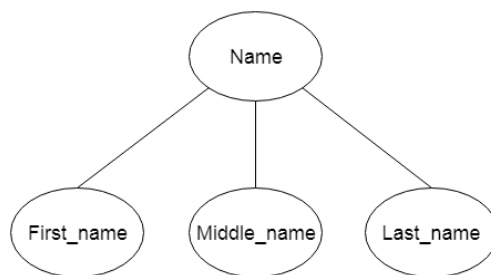
a. Key Attribute

The key attribute is used to represent the main characteristics of an entity. It represents a primary key. The key attribute is represented by an ellipse with the text underlined.



b. Composite Attribute

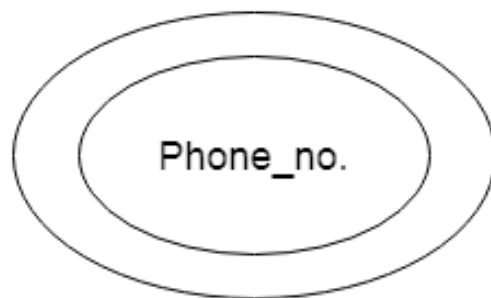
An attribute that composed of many other attributes is known as a composite attribute. The composite attribute is represented by an ellipse, and those ellipses are connected with an ellipse.



c. Multivalued Attribute

An attribute can have more than one value. These attributes are known as a multivalued attribute. The double oval is used to represent multivalued attribute.

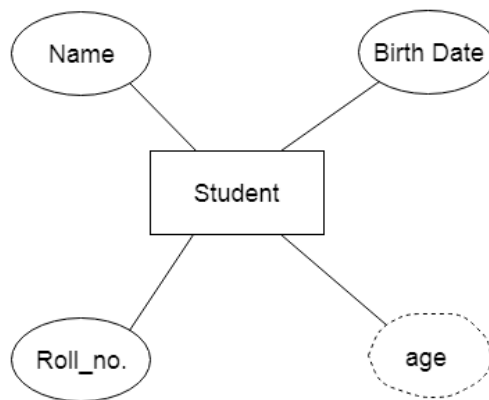
For example, a student can have more than one phone number.



d. Derived Attribute

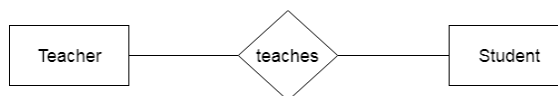
An attribute that can be derived from other attribute is known as a derived attribute. It can be represented by a dashed ellipse.

For example, A person's age changes over time and can be derived from another attribute like Date of birth.



▼ Relationship

A relationship is a connection between two or more entities. For example, a person works for a company, or an event takes place at a location. Each relationship is represented by a diamond in an ER diagram. Name of the relationship is written inside the diamond-box. All the entities (rectangles) participating in a relationship, are connected to it by a line.



Types of relationship are as follows:

a. One-to-One Relationship

When only one instance of an entity is associated with the relationship, then it is known as one to one relationship.

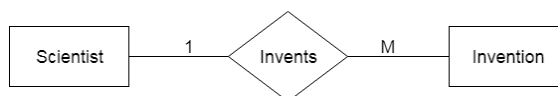
For example, A female can marry to one male, and a male can marry to one female.



b. One-to-many relationship

When only one instance of the entity on the left, and more than one instance of an entity on the right associates with the relationship then this is known as a one-to-many relationship.

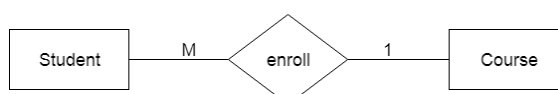
For example, Scientist can invent many inventions, but the invention is done by the only specific scientist.



c. Many-to-one relationship

When more than one instance of the entity on the left, and only one instance of an entity on the right associates with the relationship then it is known as a many-to-one relationship.

For example, Student enrolls for only one course, but a course can have many students.



d. Many-to-many relationship

When more than one instance of the entity on the left, and more than one instance of an entity on the right associates with the relationship then it is known as a many-to-many relationship.

For example, Employee can assign by many projects and project can have many employees.

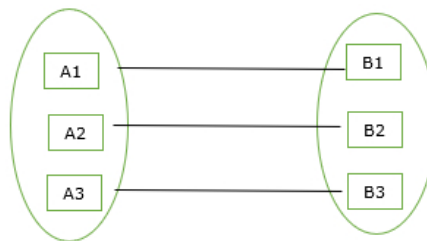


▼ Mapping Cardinality

It is expressed as the number of entities to which another entity can be associated via a relationship set.

▼ One-to-one

One entity of A is associated with one entity of B.



Example

Given below is an example of the one-to-one relationship in the mapping cardinality. Here, one department has one head of the department (HOD).

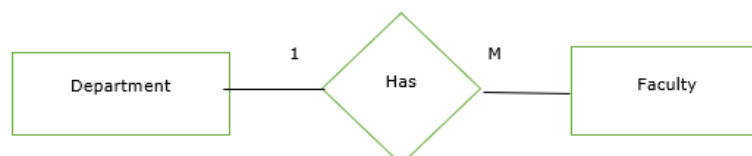


▼ One-to-many

An entity set A is associated with any number of entities in B with a possibility of zero and entity in B is associated with at most one entity in A

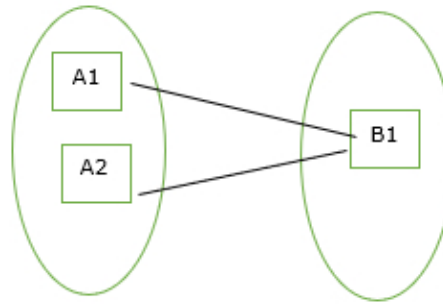
Example

Given below is an example of the one-to-many relationship in the mapping cardinality. Here, one department has many faculties.



▼ Many-to-one

An entity set A is associated with at most one entity in B and an entity set in B can be associated with any number of entities in A with a possibility of zero.



Example

Given below is an example of the many-to-one relationship in the mapping cardinality. Here, many faculties work in one department.

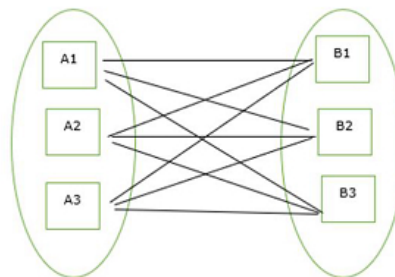


▼ Many-to-many

Many entities of A are associated with many entities of B.

An entity in A is associated with many entities of B and an entity in B is associated with many entities of A.

Many to many=many to one + one to many



Example

Given below is an example of the many-to-many relationship in the mapping cardinality. Here, many employees work on many projects.



▼ Primary key

A primary key is a field in a table that is used to uniquely identify each row in the table. It is the most important field in a table and it should not contain NULL values. One of the common examples of the primary key is an ID number, such as a student ID number, employee ID number, or product ID number.

For example, consider a table for student information. The table might have fields such as the student's first name, last name, address, and phone number. The primary key field would be a student ID number, which is used to uniquely identify each student in the table.

Student ID	First Name	Last Name	Address	Phone Number
1	John	Smith	123 Main Street	888-888-8888
2	Jane	Doe	456 Main St	999-999-9999

▼ Removing redundant attributes in Entity sets

Redundant attributes are those attributes which are not necessary to describe the relationship between two entities. Redundant attributes can lead to many problems such as data inconsistency, incorrect data and incorrect interpretation of data.

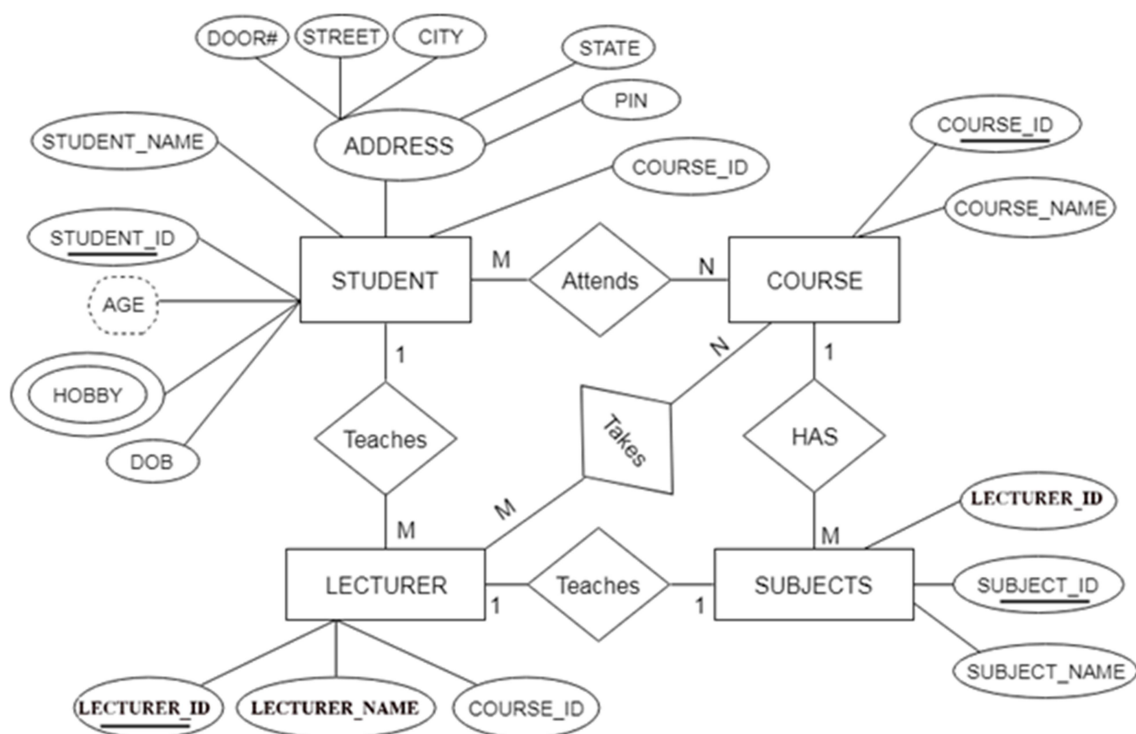
For example, consider a table for student information which has fields such as the student's first name, last name, address, and phone number. First name and last name are redundant attributes because they are not necessary to describe the relationship between student and address.

Student ID	Address	Phone Number
1	123 Main Street	888-888-8888
2	456 Main St	999-999-9999

▼ Reducing ER diagram to Relational schema

The database can be represented using the notations, and these notations can be reduced to a collection of tables.

In the database, every entity set or relationship set can be represented in tabular form.



There are some points for converting the ER diagram to the table:

- **Entity type becomes a table.**

In the given ER diagram, LECTURE, STUDENT, SUBJECT and COURSE forms individual tables.

- **All single-valued attribute becomes a column for the table.**

In the STUDENT entity, STUDENT_NAME and STUDENT_ID form the column of STUDENT table. Similarly, COURSE_NAME and COURSE_ID form the column of COURSE table and so on.

- **A key attribute of the entity type represented by the primary key.**

In the given ER diagram, COURSE_ID, STUDENT_ID, SUBJECT_ID, and LECTURE_ID are the key attribute of the entity.

- **The multivalued attribute is represented by a separate table.**

In the student table, a hobby is a multivalued attribute. So it is not possible to represent multiple values in a single column of STUDENT table. Hence we create a table STUD_HOBBY with column name STUDENT_ID and HOBBY. Using both the column, we create a composite key.

- **Composite attribute represented by components.**

In the given ER diagram, student address is a composite attribute. It contains CITY, PIN, DOOR#, STREET, and STATE. In the STUDENT table, these attributes can merge as an individual column.

-

- **Derived attributes are not considered in the table.**

In the STUDENT table, Age is the derived attribute. It can be calculated at any point of time by calculating the difference between current date and Date of Birth.

Using these rules, you can convert the ER diagram to tables and columns and assign the mapping between the tables. Table structure for the given ER diagram is as below:

▼ Entity relationship design issue

- Incorrect data models can lead to data inconsistencies and incorrect interpretations of data

- Poorly designed ER diagrams can lead to data redundancy and data duplication
- Too many relationships can lead to complex and confusing diagrams
- Poorly defined relationships can lead to inaccurate data
- Poorly defined attributes can lead to incomplete information and data integrity issues
- Unnormalized data models can lead to data integrity issues and can make it difficult to maintain database integrity
- Lack of referential integrity can lead to incorrect data retrieval and manipulation
- Poorly designed security models can lead to unauthorized access to sensitive data

▼ Relational Database Design

▼ Features of good relation design

When designing a relational database model, there are certain features that should be taken into consideration to ensure it is a well-designed and efficient database system.

These features include:

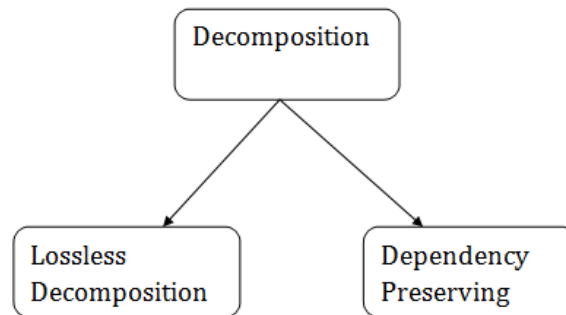
- **Normalization:** Normalization is the process of organizing data in a database to reduce data redundancy and improve data integrity. Normalization helps eliminate data anomalies and ensure the database is free of data inconsistency.
- **Referential integrity:** Referential integrity is the concept of ensuring data integrity by ensuring the relationships between entities are correctly represented. This can be done by creating foreign key constraints that enforce referential integrity.
- **Entity integrity:** Entity integrity is the concept of ensuring that no two entities have the same key. This is done by creating a primary key constraint that ensures the uniqueness of each entity.
- **Structured query language (SQL):** SQL is the language used to interact with a relational database. It is used to query, insert, update and delete data from the database. It is important to ensure that the SQL queries are optimized for speed and accuracy.
- **Security:** Security is an important aspect of database design. It is important to ensure that the database is secure from unauthorized access and manipulation. This can be done by implementing user authentication and authorization, as well as encrypting data.

These features are important to consider when designing a database model to ensure optimal performance, data integrity, and security. In addition, it is important to consider how the database will be used and by whom. This helps determine the best practices and design principles that will be used to create the database. It is also important to ensure that the SQL queries used to interact with the database are optimized for speed and accuracy. Finally, it is important to ensure that the database is secure from unauthorized access and manipulation.

▼ Decomposition using functional Dependencies

- When a relation in the relational model is not in appropriate normal form then the decomposition of a relation is required.
- In a database, it breaks the table into multiple tables.
- If the relation has no proper decomposition, then it may lead to problems like loss of information.
- Decomposition is used to eliminate some of the problems of bad design like anomalies, inconsistencies, and redundancy.

Types of Decomposition



▼ Lossless Decomposition

- If the information is not lost from the relation that is decomposed, then the decomposition will be lossless.
- The lossless decomposition guarantees that the join of relations will result in the same relation as it was decomposed.
- The relation is said to be lossless decomposition if natural joins of all the decomposition give the original relation.

Example:

EMPLOYEE_DEPARTMENT table:

EMP_ID	EMP_NAME	EMP_AGE	EMP_CITY	DEPT_ID	DEPT_NAME
22	Denim	28	Mumbai	827	Sales
33	Alina	25	Delhi	438	Marketing
46	Stephan	30	Bangalore	869	Finance
52	Katherine	36	Mumbai	575	Production
60	Jack	40	Noida	678	Testing

The above relation is decomposed into two relations EMPLOYEE and DEPARTMENT

EMPLOYEE table:

EMP_ID	EMP_NAME	EMP_AGE	EMP_CITY
22	Denim	28	Mumbai
33	Alina	25	Delhi
46	Stephan	30	Bangalore
52	Katherine	36	Mumbai
60	Jack	40	Noida

DEPARTMENT table

DEPT_ID	EMP_ID	DEPT_NAME
827	22	Sales
438	33	Marketing
869	46	Finance
575	52	Production
678	60	Testing

Now, when these two relations are joined on the common column "EMP_ID", then the resultant relation will look like:

Employee ⇄ Department

EMP_ID	EMP_NAME	EMP_AGE	EMP_CITY	DEPT_ID	DEPT_NAME
22	Denim	28	Mumbai	827	Sales
33	Alina	25	Delhi	438	Marketing
46	Stephan	30	Bangalore	869	Finance
52	Katherine	36	Mumbai	575	Production
60	Jack	40	Noida	678	Testing

Hence, the decomposition is Lossless join decomposition.

▼ Dependency Preserving

Dependency Preserving is a type of decomposition where all the functional dependencies in the original relation are preserved in the decomposed relations. This type of decomposition ensures that all the data related to a particular functional dependency is stored in the same relation.

For example, consider the EMPLOYEE_DEPARTMENT table which has the functional dependency $EMP_ID \rightarrow \{EMP_NAME, EMP_AGE, EMP_CITY, DEPT_ID, DEPT_NAME\}$.

This functional dependency can be preserved by decomposing the table into two relations, EMPLOYEE and DEPARTMENT. The EMPLOYEE table will contain all attributes related to the EMP_ID functional dependency, and the DEPARTMENT table will contain all attributes related to the DEPT_ID functional dependency. This will ensure that all the data related to the EMP_ID and DEPT_ID are stored in the same relation and all the functional dependencies are preserved in the decomposed relations.

▼ Normal forms

Normal forms are the rules used to ensure that the data stored in a relational database is organized in a manner that is efficient and consistent. Normal forms involve the decomposition of data into smaller and more manageable tables, and the use of keys to ensure data integrity.

There are three types of normal forms:

- **First Normal Form (1NF):** First normal form is the most basic form of normalization and is used to eliminate any duplicate or redundant data. It requires that each table should contain only one value per column and that each row should have a unique identifier (primary key).

ID	Name	Age	City
1	John	25	London
2	Jane	30	Paris
3	Bob	28	New York

This table is in First Normal Form because it contains only one value per column and each row has a unique identifier (ID).

- **Second Normal Form (2NF):** Second normal form is used to ensure that all non-key attributes are dependent on the entire primary key. This means that each non-key attribute must depend on the entire primary key and not just a part of it.

For example, consider a table for student information which has fields such as the student's first name, last name, address, phone number and student ID. The student ID is the primary key for this table.

Student ID	First Name	Last Name	Address	Phone Number
1	John	Smith	123 Main Street	888-888-8888
2	Jane	Doe	456 Main St	999-999-9999

In this example, the student ID is the primary key and the other fields (first name, last name, address and phone number) are non-key attributes. In order to ensure that this table is in Second Normal Form, all of the non-key attributes must depend on the entire primary key (student ID). This means that the student ID must be used to uniquely identify each row in the table.

- **Third Normal Form (3NF):** Third normal form is used to ensure that all non-key attributes are independent of each other. This means that each non-key attribute must depend only on the primary key and not on any other non-key attribute.

An example of a table in Third Normal Form is a student information table which has fields such as the student's first name, last name, address, phone number and student ID. The student ID is the primary key for this table.

Student ID	First Name	Last Name	Address	Phone Number
1	John	Smith	123 Main Street	888-888-8888
2	Jane	Doe	456 Main St	999-999-9999

In order to ensure that this table is in Third Normal Form, all of the non-key attributes (first name, last name, address and phone number) must be independent of each other. This means that each non-key attribute must depend only on the primary key (student ID) and not on any other non-key attribute.

Normal forms help to ensure the data stored in a relational database is organized in an efficient and consistent manner. By using the normal forms, it is possible to reduce data redundancy and improve data integrity.